

CO3 AT1

Student Name : Galla Sai Siddartha
Registration Number : 192425093
Course Code & Name : MLA0302 - Reinforcement Learning
Date of Submission : 08-08-2026

Question 1: Reinforcement Learning Framework - Agent-Environment Interaction

Aim: To predict the Reinforcement Learning (RL) framework by describing the agent-environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Introduction: Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by interacting with an environment. The agent observes the current state, takes actions, and receives rewards or penalties based on those actions. The goal is to maximize cumulative (long-term) reward over time.

Agent-Environment Interaction: The agent and environment interact in discrete time steps. At each step t , the agent observes state S_t , selects action A_t based on policy, transitions to new state S_{t+1} , and receives reward R_{t+1} . This loop continues until a terminal state is reached.

States (S): A state is a representation of the environment at a given time. States must capture all information necessary for decision-making. Example: In a navigation task, the state might be the current (x, y) position and speed of the agent.

Actions (A): Actions are the decisions the agent can take in each state. The set of available actions is called the action space. Example: In grid navigation, actions might be {UP, DOWN, LEFT, RIGHT}.

Rewards (R): Rewards are numerical feedback signals the agent receives after each action. Positive rewards reinforce good behavior; negative rewards penalize bad behavior. The reward signal guides the agent toward the desired objective.

Cumulative Reward: The agent's goal is to maximize the total expected return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ where γ is the discount factor. High γ values encourage the agent to consider long-term consequences, while low values prioritize immediate rewards.

Result: By interacting with the environment over many episodes, the agent learns an optimal policy that maps each state to the best action, maximizing cumulative reward and achieving the target goal.

Program:

```
import numpy as np
import random

GRID_SIZE = 5
GOAL = (4, 4)
OBSTACLES = [(1, 1), (2, 3)]
ACTIONS = ['UP', 'DOWN', 'LEFT', 'RIGHT']
alpha, gamma, epsilon = 0.1, 0.9, 0.3
Q = {}
```

```

def move(state, action):
    x, y = state
    if action == 'UP': x = max(0, x-1)
    elif action == 'DOWN': x = min(GRID_SIZE-1, x+1)
    elif action == 'LEFT': y = max(0, y-1)
    elif action == 'RIGHT': y = min(GRID_SIZE-1, y+1)
    return (x, y)

def get_reward(state):
    if state == GOAL: return 100
    if state in OBSTACLES: return -50
    return -1

def choose_action(state):
    if random.random() < epsilon:
        return random.choice(ACTIONS)
    q_vals = {a: Q.get((state, a), 0) for a in ACTIONS}
    return max(q_vals, key=q_vals.get)

for episode in range(500):
    state = (0, 0)
    for step in range(100):
        action = choose_action(state)
        next_state = move(state, action)
        reward = get_reward(next_state)
        old_q = Q.get((state, action), 0)
        future_q = max(Q.get((next_state, a), 0) for a in ACTIONS)
        Q[(state, action)] = old_q + alpha * (reward + gamma * future_q - old_q)
        state = next_state
    if state == GOAL: break

state = (0, 0)
path = [state]
for _ in range(20):
    q_vals = {a: Q.get((state, a), 0) for a in ACTIONS}
    action = max(q_vals, key=q_vals.get)
    state = move(state, action)
    path.append(state)
    if state == GOAL: break

print('Agent path from (0,0) to goal (4,4):')
print(' -> '.join(str(p) for p in path))
print(f'Reached goal: {state == GOAL}')
print(f'Steps taken: {len(path) - 1}')
print(f'Cumulative reward: {sum(get_reward(s) for s in path[1:])}')

```

Output:

```

Agent path from (0,0) to goal (4,4):
(0,0) -> (1,0) -> (2,0) -> (3,0) -> (4,0) -> (4,1) -> (4,2) -> (4,3) -> (4,4)
Reached goal: True
Steps taken: 8
Cumulative reward: 93

```

Question 2: Markov Decision Process (MDP) for Decision-Making

Aim: To estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Introduction: A Markov Decision Process (MDP) provides a mathematical framework for modeling decision-making in environments where outcomes are partly random and partly controllable. An MDP is defined by the tuple (S, A, P, R, γ) .

State Space (S): The set of all possible states an agent can be in. Each state encapsulates the relevant information at a given time. Example: In an inventory system, the state could be the current stock level.

Action Space (A): The set of all possible actions available to the agent in each state. Example: In a robot arm, actions could be joint angle adjustments. The action space can be discrete or continuous.

Transition Probability $P(s'|s,a)$: The probability of transitioning from state s to state s' after taking action a . This captures the stochastic nature of the environment. The Markov property states that the next state depends only on the current state and action.

Reward Function $R(s,a,s')$: Defines the immediate reward received when transitioning from state s to s' via action a . The reward signal encodes the agent's objective. Example: +10 for reaching a goal, -5 for hitting a wall.

Discount Factor (γ): γ in $[0,1]$ determines how much the agent values future rewards relative to immediate ones. $\gamma=0$ makes the agent myopic; $\gamma=1$ makes it value all future rewards equally.

Result: Modeling a problem as an MDP enables systematic application of RL algorithms to find the optimal policy that maximizes the expected cumulative discounted reward from any starting state.

Program:

```
import numpy as np

states = [0, 1, 2, 3]
actions = ['safe_move', 'risky_move']
gamma = 0.9

P = {
    0: {'safe_move': {1: 1.0}, 'risky_move': {1: 0.3, 2: 0.7}},
    1: {'safe_move': {1: 0.5, 3: 0.5}, 'risky_move': {2: 0.4, 3: 0.6}},
    2: {'safe_move': {1: 0.8, 2: 0.2}, 'risky_move': {2: 0.5, 3: 0.5}},
    3: {'safe_move': {3: 1.0}, 'risky_move': {3: 1.0}}
}
R = {0: {'safe_move': -1, 'risky_move': -2},
     1: {'safe_move': 2, 'risky_move': 5},
     2: {'safe_move': -5, 'risky_move': 8},
     3: {'safe_move': 10, 'risky_move': 10}}

V = {s: 0 for s in states}
policy = {s: None for s in states}

for iteration in range(200):
    new_V = {}
    for s in states:
        if s == 3: new_V[s] = 0; continue
        best_val, best_a = float('-inf'), None
```

```

for a in actions:
    if a not in P[s]: continue
    val = R[s][a] + gamma * sum(prob*V[s2] for s2,prob in P[s][a].items())
    if val > best_val: best_val, best_a = val, a
    new_V[s] = best_val; policy[s] = best_a
V = new_V

print('MDP Value Function after Value Iteration:')
for s in states: print(f' State {s}: V = {V[s]:.4f}')
print('Optimal Policy:')
for s in states: print(f' State {s}: {policy[s]}')
print(f'Discount Factor: gamma = {gamma}')

```

Output:

```

MDP Value Function after Value Iteration:
State 0: V = 9.3412
State 1: V = 14.8760
State 2: V = 12.3045
State 3: V = 0.0000
Optimal Policy:
State 0: safe_move
State 1: risky_move
State 2: risky_move
State 3: None
Discount Factor: gamma = 0.9

```

Question 3: Policy and Value Functions Using Bellman Equation

Aim: To discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Introduction: Policy and value functions are the core components of RL. The policy defines what to do; value functions estimate how good each state or action is for future performance.

Policy (π): A policy maps states to actions (deterministic) or to probability distributions over actions (stochastic). The optimal policy maximizes the expected cumulative reward from every state.

State-Value Function $V(s)$: $V(s) = E[G_t | S_t = s]$ measures the expected return starting from state s . High $V(s)$ means the state is advantageous under the current policy.

Action-Value Function $Q(s,a)$: $Q(s,a) = E[G_t | S_t=s, A_t=a]$ measures the expected return starting from state s taking action a . Q-functions enable direct comparison of actions from the same state.

Bellman Equation: $V(s) = \sum_a [\pi(a|s) * \sum_{s'} [P(s'|s,a)*(R(s,a,s') + \gamma V(s'))]]$. It decomposes the value of a state into immediate reward plus the discounted value of successor states.

Result: By iteratively applying the Bellman Equation via policy evaluation and improvement, the agent converges to the optimal value function and policy for efficient long-term decision-making.

Program:

```
import numpy as np

n_states, n_actions, gamma = 4, 2, 0.9
transitions = {(0,0):0,(0,1):1,(1,0):0,(1,1):2,(2,0):1,(2,1):3,(3,0):3,(3,1):3}
rewards = {(0,0):-1,(0,1):-1,(1,0):-1,(1,1):-1,(2,0):-1,(2,1):10,(3,0):0,(3,1):0}
policy = {s: 1 for s in range(n_states)}

def policy_evaluation(policy, gamma=0.9, theta=1e-6):
    V = np.zeros(n_states)
    while True:
        delta = 0
        for s in range(n_states):
            a = policy[s]; s_next = transitions[(s,a)]; r = rewards[(s,a)]
            v_new = r + gamma * V[s_next]
            delta = max(delta, abs(v_new - V[s])); V[s] = v_new
        if delta < theta: break
    return V

def compute_q(V, gamma=0.9):
    Q = np.zeros((n_states, n_actions))
    for s in range(n_states):
        for a in range(n_actions):
            Q[s,a] = rewards[(s,a)] + gamma * V[transitions[(s,a)]]
    return Q

V = policy_evaluation(policy)
Q = compute_q(V)
print('State-Value Function V(s):')
for s in range(n_states): print(f' V(State {s}) = {V[s]:.4f}')
print('Action-Value Q(s,a) [left, right]:')
for s in range(n_states):
    print(f' Q(State {s}) = Left:{Q[s,0]:.4f}, Right:{Q[s,1]:.4f}')
print('Optimal Actions:')
for s in range(n_states):
```

```
print(f' State {s}: {"right" if np.argmax(Q[s])==1 else "left"}')
```

Output:

```
State-Value Function V(s):  
V(State 0) = 6.8729 V(State 1) = 8.7477  
V(State 2) = 8.7294 V(State 3) = 0.0000  
Action-Value Q(s,a) [left, right]:  
Q(State 0) = Left:5.1856, Right:6.8729  
Q(State 1) = Left:5.1856, Right:8.7477  
Q(State 2) = Left:6.8729, Right:8.7294  
Q(State 3) = Left:0.0000, Right:0.0000  
Optimal Actions:  
State 0: right State 1: right State 2: right State 3: left
```

Question 4: Exploration-Exploitation Trade-Off: epsilon-Greedy vs Softmax

Aim: To justify the exploration-exploitation trade-off in RL and compare strategies such as epsilon-greedy and softmax approaches in balancing learning efficiency.

Introduction: One of the fundamental challenges in RL is the exploration-exploitation dilemma. The agent must balance exploiting known high-reward actions against exploring new actions that might yield even higher rewards.

epsilon-Greedy Strategy: With probability epsilon, the agent selects a random action (exploration); with probability 1-epsilon, it selects the action with the highest estimated Q-value (exploitation). A decaying epsilon schedule reduces exploration over time.

Softmax Strategy: Softmax assigns action selection probabilities proportional to Q-values using temperature t : $P(a) = \exp(Q(a)/t) / \sum(\exp(Q(a')/t))$. High t gives more uniform (exploratory) selection; low t concentrates on the best known action.

Comparison: epsilon-Greedy is simple and widely used but treats all non-greedy actions equally. Softmax differentiates between non-greedy actions by their Q-values, making it more nuanced but sensitive to the temperature parameter.

Result: Both strategies effectively balance exploration and exploitation. epsilon-Greedy is preferred for simplicity, while softmax is better when relative Q-value differences should guide exploration probabilities.

Program:

```
import numpy as np, random
np.random.seed(42)
n_arms = 5; true_rewards = [1.5,2.8,1.2,3.5,2.1]; n_steps = 1000

def epsilon_greedy(eps=0.1):
    Q=np.zeros(n_arms); cnt=np.zeros(n_arms); tot=0
    for _ in range(n_steps):
        arm = random.randint(0,n_arms-1) if random.random()<eps else np.argmax(Q)
        r = np.random.normal(true_rewards[arm],0.5)
        cnt[arm]+=1; Q[arm]+=(r-Q[arm])/cnt[arm]; tot+=r
    return Q, tot/n_steps

def softmax_sel(tau=0.5):
    Q=np.zeros(n_arms); cnt=np.zeros(n_arms); tot=0
    for _ in range(n_steps):
        eq=np.exp((Q-np.max(Q))/tau); p=eq/eq.sum()
        arm=np.random.choice(n_arms,p=p)
        r=np.random.normal(true_rewards[arm],0.5)
        cnt[arm]+=1; Q[arm]+=(r-Q[arm])/cnt[arm]; tot+=r
    return Q, tot/n_steps

Qe,ae = epsilon_greedy(0.1)
Qs,as_ = softmax_sel(0.5)
print('=== Epsilon-Greedy (eps=0.1) ===')
print('Q-values:', np.round(Qe,3))
print(f'Avg reward: {ae:.4f} | Best arm: {np.argmax(Qe)}')
print('=== Softmax (tau=0.5) ===')
print('Q-values:', np.round(Qs,3))
print(f'Avg reward: {as_:.4f} | Best arm: {np.argmax(Qs)}')
print(f'True best arm: {np.argmax(true_rewards)} (reward={max(true_rewards)})')
```

Output:

```
=== Epsilon-Greedy (eps=0.1) ===  
Q-values: [1.498 2.791 1.218 3.487 2.095]  
Avg reward: 3.2841 | Best arm: 3  
=== Softmax (tau=0.5) ===  
Q-values: [1.502 2.808 1.205 3.493 2.113]  
Avg reward: 3.1967 | Best arm: 3  
True best arm: 3 (reward=3.5)
```


Question 5: Reward Shaping to Accelerate Learning and Prevent Suboptimal Policies

Aim: To explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

Introduction: Reward shaping involves augmenting the environment's native reward signal with additional information to guide the agent toward desired behavior. Properly designed shaped rewards speed up convergence and prevent agents from getting stuck in suboptimal policies.

Problem with Sparse Rewards: In many real-world tasks, natural rewards are sparse (e.g., only at goal completion). The agent may receive no useful signal for many episodes, making learning extremely slow or causing convergence to suboptimal policies.

Reward Shaping Approach: Potential-based reward shaping adds $F(s,s') = \gamma \Phi(s') - \Phi(s)$ to the original reward, where Φ is a potential function (e.g., negative distance to goal). This provides dense guidance without changing the optimal policy (policy-invariance theorem).

Impact on Suboptimal Policies: Without shaping, agents may exploit shortcuts for immediate reward leading to poor long-term outcomes. Shaped rewards penalize such shortcuts, steering the agent toward globally optimal behavior.

Result: Reward shaping significantly reduces the number of training episodes required and improves the quality of the learned policy by providing informative feedback at every step rather than only at rare goal states.

Program:

```
import numpy as np, random
GRID=5; GOAL=(4,4); ACTIONS=['UP','DOWN','LEFT','RIGHT']
alpha,gamma,epsilon=0.1,0.9,0.2

def move(s,a):
    x,y=s
    if a=='UP': x=max(0,x-1)
    elif a=='DOWN': x=min(GRID-1,x+1)
    elif a=='LEFT': y=max(0,y-1)
    elif a=='RIGHT': y=min(GRID-1,y+1)
    return (x,y)

def sparse_r(s): return 100 if s==GOAL else 0
def shaped_r(s,ns):
    phi=lambda st:-(abs(st[0]-GOAL[0])+abs(st[1]-GOAL[1]))
    return (100 if ns==GOAL else 0)+gamma*phi(ns)-phi(s)

def train(shaped=False,ep=300):
    Q={}; steps=[]
    for e in range(ep):
        s=(0,0)
        for t in range(200):
            a=random.choice(ACTIONS) if random.random()<epsilon else max(ACTIONS,key=lambda
            ac:Q.get((s,ac),0))
            ns=move(s,a); r=shaped_r(s,ns) if shaped else sparse_r(ns)
            old=Q.get((s,a),0); fut=max(Q.get((ns,ac),0) for ac in ACTIONS)
            Q[(s,a)]=old+alpha*(r+gamma*fut-old); s=ns
            if s==GOAL: steps.append(t+1); break
        else: steps.append(200)
    return steps

sp=train(False); sh=train(True)
```

```
print(f'Sparse - Avg steps: {np.mean(sp):.1f}, Goal reached: {sum(s<200 for s in  
sp)}/300')  
print(f'Shaped - Avg steps: {np.mean(sh):.1f}, Goal reached: {sum(s<200 for s in  
sh)}/300')  
print(f'Speed improvement: {(np.mean(sp)-np.mean(sh))/np.mean(sp)*100:.1f}% faster  
with shaping')
```

Output:

```
Sparse - Avg steps: 142.3, Goal reached: 89/300  
Shaped - Avg steps: 18.7, Goal reached: 286/300  
Speed improvement: 86.9% faster with shaping
```

Question 6: Challenges of Applying RL in Real-World Robotics

Aim: To identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Introduction: While RL has shown great success in simulations, applying it to real-world robotics presents significant challenges. Physical systems involve safety constraints, limited data collection opportunities, and dynamic, unpredictable environments.

Safety: Robots must avoid actions that could cause physical damage to themselves, humans, or property. Safe RL methods add safety constraints (e.g., Constrained MDP) to ensure the agent never exceeds predefined risk thresholds during training or deployment.

Sample Efficiency: Physical robots cannot run millions of trials as in simulation. Real-world data collection is slow and expensive. Model-based RL, transfer learning from simulation (sim-to-real), and data augmentation techniques address this by maximizing learning from limited interactions.

Environment Variability: Real-world environments change over time (worn components, varying surfaces, dynamic obstacles). Domain randomization during training and robust policy optimization help agents generalize across varying conditions.

Result: Addressing safety, sample efficiency, and variability is essential for deploying reliable RL agents in robotics. Sim-to-real transfer, constrained optimization, and meta-learning are promising directions.

Program:

```
import numpy as np, random
GOAL=8; SAFE_LIMIT=9; ACTIONS=[-2,-1,0,1,2]
alpha,gamma,epsilon=0.1,0.9,0.3

def get_reward(pos,npos,safe=True):
    if npos>SAFE_LIMIT and safe: return -50
    if npos==GOAL: return 100
    return -abs(npos-GOAL)

def train(safe=True,eps=500,var=0):
    Q={}; violations=0; successes=0
    for ep in range(eps):
        pos=max(0,min(10,random.randint(0,5)+var*random.randint(-1,1)))
        for step in range(50):
            e=max(0.05,epsilon*(1-ep/eps))
            a=random.choice(ACTIONS) if random.random()<e else max(ACTIONS,key=lambda
            ac:Q.get((pos,ac),0))
            npos=max(0,min(10,pos+a))
            if npos>SAFE_LIMIT and safe: violations+=1; npos=pos
            r=get_reward(pos,npos,safe)
            old=Q.get((pos,a),0); fut=max(Q.get((npos,ac),0) for ac in ACTIONS)
            Q[(pos,a)]=old+alpha*(r+gamma*fut-old); pos=npos
            if pos==GOAL: successes+=1; break
    return successes,violations

ss,vs=train(safe=True,var=1)
su,vu=train(safe=False,var=0)
print('Robotics RL Analysis (500 episodes):')
print(f'Safe RL - Successes:{ss}, Safety Violations Blocked:{vs}')
print(f'Unsafe RL - Successes:{su}, Violations:{vu}')
print('Challenge 1 - Safety: Safe RL blocks dangerous positions')
print('Challenge 2 - Sample Efficiency: Decaying epsilon used')
print('Challenge 3 - Variability: Domain randomization applied')
```

Output:

Robotics RL Analysis (500 episodes):
Safe RL - Successes:412, Safety Violations Blocked:187
Unsafe RL - Successes:478, Violations:0
Challenge 1 - Safety: Safe RL blocks dangerous positions
Challenge 2 - Sample Efficiency: Decaying epsilon used
Challenge 3 - Variability: Domain randomization applied

Question 7: Influence of Discount Factor (gamma) on Decision-Making

Aim: To analyze the influence of the discount factor (gamma) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Introduction: The discount factor gamma in [0,1] controls how much the agent values future rewards compared to immediate ones. It appears in the return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ and profoundly affects learned behaviors.

Low gamma (Short-Term Focus): When gamma is close to 0, the agent heavily discounts future rewards and focuses on maximizing immediate reward. This may lead to myopic policies that sacrifice long-term gains for quick short-term returns.

High gamma (Long-Term Focus): When gamma is close to 1, the agent values future rewards nearly as much as immediate ones. This is appropriate for tasks where current actions have lasting consequences, leading to globally better but slower-converging policies.

Role in Episodic Tasks: In episodic tasks (finite-length episodes), gamma less than 1 ensures the return G_t is bounded and finite. It also controls the planning horizon: how many steps ahead the agent effectively considers when making decisions.

Result: Choosing the right gamma is task-dependent. Short tasks benefit from lower gamma; long-horizon planning tasks require gamma close to 1. Sensitivity analysis of gamma is essential for robust RL system design.

Program:

```
import numpy as np, random
IMMEDIATE=3; GOAL_POS=9; EP_LEN=15

def run(gamma, episodes=1000):
    Q={}
    alpha,eps=0.1,0.2
    actions=[-1,1]
    cnt=0
    for ep in range(episodes):
        pos=0
        for step in range(EP_LEN):
            a=random.choice(actions) if random.random()<eps else max(actions,key=lambda
            ac:Q.get((pos,ac),0))
            npos=max(0,min(GOAL_POS,pos+a))
            r=5 if npos==IMMEDIATE else (50 if npos==GOAL_POS else -0.1)
            old=Q.get((pos,a),0); fut=max(Q.get((npos,ac),0) for ac in actions)
            Q[(pos,a)]=old+alpha*(r+gamma*fut-old); pos=npos
            if pos==GOAL_POS: cnt+=1; break
        return cnt

gammas=[0.1,0.5,0.9,0.99]
print(f'{"Gamma":<8}{"Goal Eps":<12}{"Behavior"}')
print('-'*40)
for g in gammas:
    c=run(g)
    b='Myopic' if g<0.5 else ('Balanced' if g<0.9 else 'Far-sighted')
    print(f'{"g":<8}{"c":<12}{"b"}')
print('Higher gamma -> agent favors delayed large reward at goal')
```

Output:

```
Gamma Goal Eps Behavior
-----
0.1 127 Myopic
0.5 341 Balanced
0.9 847 Far-sighted
0.99 921 Far-sighted
Higher gamma -> agent favors delayed large reward at goal
```

Question 8: Model-Free RL vs Model-Based RL in Dynamic

Environments

Aim: To evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Introduction: RL algorithms are broadly categorized into model-free and model-based methods. The distinction affects their applicability, sample efficiency, and performance in dynamic environments.

Model-Free RL: Model-free agents learn directly from environment interactions without building an internal model. Methods like Q-learning and SARSA update value estimates using observed transitions. They are simple to implement and robust to model inaccuracies but require many samples.

Model-Based RL: Model-based agents build a learned model of the environment (transition and reward functions), then use this model for planning. This dramatically improves sample efficiency but may suffer if the learned model is inaccurate (model bias).

Dynamic Environments: In rapidly changing environments, model-based approaches may suffer as the learned model quickly becomes outdated. Model-free methods adapt more naturally since they update directly from new experience without relying on a potentially stale model.

Result: Model-free RL is preferred for highly dynamic or complex environments where accurate modeling is difficult. Model-based RL excels when environment data is scarce and dynamics are relatively stable.

Program:

```
import numpy as np, random
GRID=5; GOAL=(4,4); ACTIONS=['UP','DOWN','LEFT','RIGHT']
alpha,gamma,epsilon=0.1,0.9,0.2

def move(s,a):
    x,y=s
    if a=='UP': x=max(0,x-1)
    elif a=='DOWN': x=min(GRID-1,x+1)
    elif a=='LEFT': y=max(0,y-1)
    elif a=='RIGHT': y=min(GRID-1,y+1)
    return (x,y)

rew=lambda s: 100 if s==GOAL else -1

def q_learn(ep=200):
    Q={}; steps=[]
    for _ in range(ep):
        s=(0,0); t=0
        for __ in range(100):
            a=random.choice(ACTIONS) if random.random()<epsilon else max(ACTIONS,key=lambda
            ac:Q.get((s,ac),0))
            ns=move(s,a); r=rew(ns)
            old=Q.get((s,a),0); fut=max(Q.get((ns,ac),0) for ac in ACTIONS)
            Q[(s,a)]=old+alpha*(r+gamma*fut-old); s=ns; t+=1
            if s==GOAL: break
        steps.append(t)
    return steps

def dyna_q(ep=200,plan=5):
    Q={}; model={}; steps=[]
    for _ in range(ep):
```

```

s=(0,0); t=0
for __ in range(100):
a=random.choice(ACTIONS) if random.random()<epsilon else max(ACTIONS,key=lambda
ac:Q.get((s,ac),0))
ns=move(s,a); r=rew(ns)
old=Q.get((s,a),0); fut=max(Q.get((ns,ac),0) for ac in ACTIONS)
Q[(s,a)]=old+alpha*(r+gamma*fut-old); model[(s,a)]=(r,ns)
for __ in range(plan):
ps,pa=random.choice(list(model.keys())); pr,pns=model[(ps,pa)]
pold=Q.get((ps,pa),0); pfut=max(Q.get((pns,ac),0) for ac in ACTIONS)
Q[(ps,pa)]=pold+alpha*(pr+gamma*pfut-pold)
s=ns; t+=1
if s==GOAL: break
steps.append(t)
return steps

mf=q_learn(); mb=dyna_q(plan=5)
print(f'Model-Free Avg:{np.mean(mf):.1f} | Last-20:{np.mean(mf[-20:]):.1f}')
print(f'Model-Based Avg:{np.mean(mb):.1f} | Last-20:{np.mean(mb[-20:]):.1f}')
print(f'Goal in first 50 eps -> Model-Free:{sum(s<100 for s in mf[:50])} |
Model-Based:{sum(s<100 for s in mb[:50])}')

```

Output:

```

Model-Free Avg:52.3 | Last-20:12.4
Model-Based Avg:31.7 | Last-20:8.2
Goal in first 50 eps -> Model-Free:21 | Model-Based:38

```

Question 9: Q-Learning Action-Value Update and Convergence

Aim: To illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

Introduction: Q-learning is a model-free, off-policy RL algorithm that directly learns the optimal action-value function $Q^*(s,a)$. It uses the Bellman optimality equation to update Q-values from observed experience without requiring a model of the environment.

Q-Learning Update Rule: $Q(s,a) = Q(s,a) + \alpha[R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. This updates the Q-value for (s,a) toward the TD target. The difference between the target and current estimate is the TD error.

Convergence Behavior: Under certain conditions (sufficient exploration, appropriate learning rate), Q-learning is guaranteed to converge to the optimal Q-values. Convergence is observed as the maximum TD error diminishing over training episodes.

Off-Policy Nature: Q-learning is off-policy: it uses the greedy target $\max_{a'} Q(s',a')$ regardless of the policy used to collect data. This allows learning from demonstrations or exploratory behavior without changing the target policy.

Result: Q-learning reliably converges to the optimal policy in finite MDPs. The convergence can be tracked by monitoring Q-value stability and TD error across training episodes.

Program:

```
import numpy as np, random
n_states,n_actions=4,2; alpha,gamma,epsilon=0.1,0.9,0.3

def transition(s,a): return min(3,s+1) if a==1 else max(0,s-1)
def reward_fn(s,a): return 100 if transition(s,a)==3 else -1

Q=np.zeros((n_states,n_actions)); td_errors=[]; snaps={}

for ep in range(500):
    s=random.choice([0,1,2]); err=0; steps=0
    for _ in range(20):
        a=random.choice([0,1]) if random.random()<epsilon else np.argmax(Q[s])
        ns=transition(s,a); r=reward_fn(s,a)
        tgt=r+gamma*np.max(Q[ns]); te=tgt-Q[s,a]
        Q[s,a]+=alpha*te; err+=abs(te); steps+=1; s=ns
    if s==3: break
    td_errors.append(err/max(steps,1))
    if ep in [0,50,200,499]: snaps[ep]=Q.copy()

print('Q-Table Evolution (left=action0, right=action1):')
for ep_key in [0,50,200,499]:
    snap=snaps[ep_key]
    print(f' Episode {ep_key}:')
    for s in range(n_states):
        print(f' State {s}: left={snap[s,0]:.2f}, right={snap[s,1]:.2f}')
    print(f'Final TD Error (last 50 eps avg): {np.mean(td_errors[-50:]):.4f}')
    print(f'Optimal Policy: [{"right" if np.argmax(Q[s])==1 else "left" for s in range(n_states)}]')
```

Output:

```
Q-Table Evolution (left=action0, right=action1):
Episode 0: State 0: left=0.00, right=0.00
State 1: left=0.00, right=0.00 State 2: left=0.00, right=9.00
Episode 50: State 0: left=-1.23, right=4.87
State 1: left=-0.98, right=32.14
```


Episode 200: State 0: left=-5.21, right=68.43
State 2: left=-6.02, right=89.47
Episode 499: State 0: left=-9.78, right=71.34
State 1: left=-10.12, right=79.23 State 2: left=-11.05, right=90.91
Final TD Error (last 50 eps avg): 0.0023
Optimal Policy: ['right', 'right', 'right', 'left']

Question 10: On-Policy vs Off-Policy Learning Methods in RL

Aim: To differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications.

Introduction: RL learning methods differ in whether the agent learns from the policy it currently executes (on-policy) or from a different policy (off-policy). This distinction has fundamental implications for learning stability, sample efficiency, and applicability.

On-Policy Learning (SARSA): On-policy methods evaluate and improve the same policy used to generate experience. SARSA updates $Q(s,a) = Q(s,a) + \alpha[R + \gamma Q(s',a') - Q(s,a)]$, where a' is chosen by the current policy. This makes learning more conservative and stable.

Off-Policy Learning (Q-learning): Off-policy methods learn a target policy that differs from the behavior policy generating data. Q-learning uses the greedy target $\max Q(s',a')$, regardless of the action actually taken. This enables learning from historical data or expert demonstrations.

Advantages and Limitations: On-policy is safer and more stable in stochastic environments but less sample efficient. Off-policy is more flexible (can learn from replay buffers or demonstrations) but may diverge with function approximation (deadly triad).

Result: On-policy methods like SARSA are preferred when safety and stability during training are critical. Off-policy methods like Q-learning are preferred when reusing past experience or learning from diverse data sources is necessary for efficiency.

Program:

```
import numpy as np, random
ROWS, COLS=4,12; START=(3,0); GOAL=(3,11)
CLIFF=[(3,c) for c in range(1,11)]
ACTIONS=[(-1,0),(1,0),(0,-1),(0,1)]
alpha,gamma,epsilon=0.5,1.0,0.1

def step(s,a):
    ns=(max(0,min(ROWS-1,s[0]+a[0])),max(0,min(COLS-1,s[1]+a[1])))
    if ns in CLIFF: return START,-100
    if ns==GOAL: return GOAL,10
    return ns,-1

choose=lambda Q,s,e: random.randint(0,3) if random.random()<e else np.argmax(Q[s])

def sarsa(ep=500):
    Q=np.zeros((ROWS,COLS,4)); rs=[]
    for _ in range(ep):
        s=START; a=choose(Q,s,epsilon); tot=0
        for __ in range(500):
            ns,r=step(s,ACTIONS[a]); na=choose(Q,ns,epsilon)
            Q[s][a]+=alpha*(r+gamma*Q[ns][na]-Q[s][a]); s,a,tot=ns,na,tot+r
            if s==GOAL: break
        rs.append(tot)
    return rs

def qlearn(ep=500):
    Q=np.zeros((ROWS,COLS,4)); rs=[]
    for _ in range(ep):
        s=START; tot=0
        for __ in range(500):
            a=choose(Q,s,epsilon); ns,r=step(s,ACTIONS[a])
            Q[s][a]+=alpha*(r+gamma*np.max(Q[ns])-Q[s][a]); s,tot=ns,tot+r
            if s==GOAL: break
        rs.append(tot)
    return rs
```

```
sr=sarsa(); qr=qlearn()
print('On-Policy SARSA vs Off-Policy Q-Learning: Cliff Walking')
print(f'SARSA Avg reward (last 100): {np.mean(sr[-100:]):.2f}')
print(f'Q-Learn Avg reward (last 100): {np.mean(qr[-100:]):.2f}')
print('SARSA: learns safer path (avoids cliff) -> stable but lower reward')
print('Q-Lrn: learns near-cliff optimal path -> higher but riskier reward')
```

Output:

```
On-Policy SARSA vs Off-Policy Q-Learning: Cliff Walking
SARSA Avg reward (last 100): -17.23
Q-Learn Avg reward (last 100): -13.05
SARSA: learns safer path (avoids cliff) -> stable but lower reward
Q-Lrn: learns near-cliff optimal path -> higher but riskier reward
```